Modeling Tweet Virality with BERTweet Embeddings and Metadata

Sahana Savarkar

December 12, 2025

Abstract

Predicting the virality of social media content remains a fundamental challenge in computational social science and natural language processing. This study investigates the extent to which linguistic content alone can predict tweet virality by leveraging BERTweet embeddings. Using a dataset of 248,915 English-language tweets labeled with three ordinal virality classes (low, medium, high), we train and compare two predictive models: a text-only classifier using pure linguistic features, and a hybrid model combining these embeddings with six metadata features. Our text-only model substantially outperforms random baselines (see Section 10), demonstrating that linguistic content carries measurable predictive signal. The hybrid model improves overall accuracy, primarily by better identifying non-viral tweets, though performance on high-virality content remains limited. Analysis reveals that while text-based features successfully distinguish ordinary tweets from viral ones, structural amplification mechanisms not captured by text or basic metadata are necessary to reliably predict highly viral content. These findings highlight the promise as well as the limits of language-based virality prediction, and they pave the way for future work to integrate network-level features and multimodal content to achieve more robust performance across all virality classes.

1 Introduction

Social media has fundamentally altered how people obtain news and develop opinions about current events. Ideas, hashtags, and memes can reach millions of people within seconds on platforms like X (formerly, and for the purposes of this research, known as Twitter), reaching the status of "virality." Predicting the virality of online content has thus become an active research area in computational social science, marketing analytics, and natural-language processing (NLP). Our central question is this: to what extent can the *pure linguistic content* of a tweet predict its eventual reach, relative to—or in combination with—the metadata typically associated with online engagement?

This project investigates whether BERTweet embeddings, when combined with lightweight classifiers, can reliably predict tweet virality. We formalize virality as a three-class ordinal variable (low, medium, high) based on retweet and quote-tweet counts. Each tweet is

mapped into a 768-dimensional embedding via BERTweet, producing a fixed-length representation of its linguistic content. To evaluate the predictive contribution of text alone, we first train a text-only classifier that uses only these embeddings. We then extend the model to a *hybrid* variant that augments the embeddings with six interpretable metadata features (follower count, likes, retweets, quotes, and positive/negative sentiment scores), allowing us to directly compare the relative predictive value of linguistic information and lightweight structural signals.

Our goals are as follows: to evaluate the extent to which semantic and stylistic features of tweet text alone can explain virality, and to assess how much additional predictive power is gained when incorporating basic metadata features that are readily available in most Twitter-derived datasets. As discussed in Section 4, our findings show that linguistic content contains a measurable but limited signal: the text-only classifier achieves test accuracies exceeding 68%, substantially outperforming random baselines but struggling to distinguish medium- and high-virality tweets. The hybrid model yields modest but consistent improvements across metrics, suggesting that while text is informative, metadata remains an important complementary source of signal. Taken together, these results highlight both the promise and the boundaries of text-based virality prediction in large-scale social-media settings.

2 Background

In prior research, some authors define virality based on a predetermined retweet threshold (6), whereas others may define virality based on how many retweets a tweet generates relative to other tweets in the dataset (7). This section situates our work within existing machine-learning literature on classification tasks, transformer-based text representations, and the empirical study of virality on platforms such as Twitter.

We recognize the potential challenges of modeling tweet text: social-media language is informal, multimodal, and structurally inconsistent, which can make it difficult to extract meaningful features when using traditional NLP pipelines. Today, state-of-the-art transformer models trained directly on Twitter data can capture stylistic, semantic, and cultural nuances that characterize platform-specific communication. BERTweet stands out as the first large-scale RoBERTa-based language model pre-trained on an 80GB corpus comprising 850 million English tweets, yielding performance improvements over generic BERT-style encoders on core NLP tasks such as part-of-speech tagging, named-entity recognition, and text classification (2).

2.1 Classification Tasks in Machine Learning

Machine learning tasks are often divided into two major categories: classification and regression. In a *classification* task, an algorithm learns a function

$$f : \mathbb{R}^n \to \{1, \ldots, k\},$$

that assigns each input vector $x \in \mathbb{R}^n$ to one of k discrete categories (3). Modern classification models often output a full probability distribution over classes via a softmax layer, particularly in neural-network settings.

In contrast, *regression* tasks ask the model to predict a continuous output $y \in \mathbb{R}$. Virality prediction has historically been framed as a regression problem over engagement counts (retweets and impressions), but more recent work treats it as a *multiclass classification* problem due to the heavy-tailed nature of engagement distributions on social media.

Our formulation follows the latter approach: we start by discretizing virality into three ordinal levels (low, medium, high), and then learn a multiclass classifier that predicts a tweet's virality category from its text (and, in the hybrid model, additional metadata).

2.2 Predicting Virality on Social Media

Prior work informs us that large-scale diffusion is heavily shaped by structural amplification mechanisms as opposed to simply linguistic properties. Nevertheless, a parallel line of work has shown that linguistic features, such as sentiment, novelty, emotional tone, humor, or political framing, carry measurable predictive signal (8; 9). As a result of the development of specialized contextual language models such as BERTweet, researchers aiming to predict virality are equipped to capture an unprecedented level of stylistic subtlety in textual content, and succeed in areas where traditional bag-of-words methods fail in capturing these subtleties.

Our study contributes to this line of research by isolating the linguistic components of virality via BERTweet embeddings, then comparing performance against a hybrid model that incorporates six commonly studied metadata features (followers, likes, retweets, quotes, positive sentiment, negative sentiment).

2.3 Transformer Models and Tweet Embeddings

Transformer models become the dominant architecture in NLP due to their ability to model long-range dependencies through multi-head self-attention (4). Pretrained transformer encoders produce contextualized token embeddings that are sensitive to syntax, semantics, and discourse context, enabling strong performance on a wide range of downstream tasks.

For Twitter-specific language, generic pretrained models such as BERT or RoBERTa are suboptimal because social-media text differs substantially from standard corpora in terms of abbreviations, emojis, and platform-specific linguistic conventions. To address this, Nguyen et al. (2) introduced **BERTweet**, the first large-scale RoBERTa-based language model trained on an 80GB corpus of 850 million English tweets. BERTweet has been shown to outperform $\text{RoBERTa}_{\text{base}}$ and $\text{XLM-R}_{\text{base}}$ on part-of-speech tagging, named-entity recognition, and tweet-level text classification.

Given these advantages, we adopt BERTweet as the foundation of our feature extraction pipeline. Each tweet is mapped into a 768-dimensional embedding, which serves as a dense semantic representation of its content. These embeddings allow us to test how much "pure text" contributes to virality prediction, while maintaining compatibility with transformer-based hybrid models.

2.4 Metadata and Hybrid Modeling

Although text carries meaningful signals related to sentiment, tone, and topicality, most virality variance is often explained by metadata rather than linguistic features. Prior studies consistently identify follower count, posting time, early-engagement indicators, and historical user activity as highly predictive of diffusion size (10; 11). Hybrid modeling—combining text with metadata—typically yields the strongest performance.

In light of this, our study evaluates two distinct models:

1. A **text-only model** using BERTweet embeddings as inputs to a lightweight classifier.
2. A **hybrid model** augmenting these embeddings with six auxiliary metadata features.

By comparing these models on identical train–test splits, we quantify how much additional predictive power metadata provides beyond linguistic content alone. This dual-model framework allows us to better understand the respective contributions of textual and structural factors to tweet virality.

3 Methods

3.1 Data & Preprocessing

We use 248,915 English-language tweets, each labeled with one of three virality classes. Before modeling, all tweets undergo a lightweight cleaning procedure consisting of whitespace normalization using the `.strip()` method. We deliberately retain features such as URLs, hashtags, and emojis, to preserve the linguistic cues relevant to virality on Twitter, but this necessitates file encoding, which was automatically inferred using the `chardet` library. We loaded the dataset using `pandas` with the detected encoding to ensure that all Unicode characters were preserved correctly.

To accommodate variable dataset schemas, the preprocessing function dynamically detected the text column by scanning for common names `["text", "tweet", "content", "body", "message"]`. If none were found, the column could be specified manually. This enabled consistent downstream processing across heterogeneous datasets, and ensured that the pipeline would not break on differently formatted data.

3.1.1 Minimal Tweet-Compatible Text Cleaning

Text cleaning was deliberately minimal to preserve the linguistic and structural features of tweets. The BERTweet model was trained on raw Twitter data and therefore expects the presence of hashtags, user mentions, emojis, URLs, and colloquial orthography. Only leading and trailing whitespace was removed. The cleaned text was stored in a new column (`clean_text`).

3.2 Defining Virality

3.2.1 Virality Score

A continuous virality score was defined as the sum of retweets and quote tweets, representing total public amplification of a given tweet:

```
virality_raw = retweets + quotes
```

3.2.2 Discrete Virality Classes

The goal was to predict virality as a three-class ordinal variable:

- 0: low virality
- 1: medium virality
- 2: high virality.

$\leq$ 70th percentile $\rightarrow$ Low, 70th–90th percentile $\rightarrow$ Medium, $\geq$ 90th percentile $\rightarrow$ High.

3.3 Tokenizing

Each tweet is first preprocessed and then converted into a sequence of subword tokens using the BERTweet tokenizer.
Formally, for a tweet i, the tokenizer defines a mapping

$$\text{token} : T \rightarrow V^{k_i}$$

where T is the space of raw text strings and V^{k_i} is the tokenizer vocabulary. To obtain fixed-dimensional representations of tweet text, each input tweet was first tokenized and passed through the BERTweet transformer. For a given tweet i the tokenizer produces an ordered sequence of k_i tokens:

$$X_i = (x_{i,1}, x_{i,2}, \ldots, x_{i,k_i})$$

To ensure uniform batch shapes during model training, each token sequence is forced to a fixed length of 64 tokens—sequences longer than 64 tokens are truncated, while shorter sequences are padded, both from the right—following the implementation:

```
max_length=64,
truncation=True,
padding="max_length"
```

When we pass a tweet (a sequence of tokens) into BERTweet, the model outputs a vector for each individual token. The BERTweet tokenizer preserves Twitter-specific linguistic patterns using tokens, including usernames (e.g., @user), URLs (httpURL), emojis and emoticons, hashtags, and common Twitter abbreviations and colloquialisms (such as "OOMF": "one of my followers" and "IDK": "I don't know"). This design is crucial to our model as it provides more faithful representations of social-media language than generic BERT tokenizers.

3.4 Extracting Embeddings

BERTweet then maps each token to a contextualized hidden-state vector, yielding

$$H_i = (\mathbf{h}_{i,1}, \mathbf{h}_{i,2}, \ldots, \mathbf{h}_{i,k_i})$$

where each $\mathbf{h}_{i,j} \in \mathbb{R}^{768}$ corresponds to a contextualized embedding for token j. Thus, the model produces a sequence of representations in $\mathbb{R}^{k_i \times 768}$, one for each token in the tweet.

However, since downstream classifiers require fixed-dimensional inputs, we must aggregate the variable-length token sequence into a single vector representation, thus we create a mapping from $\mathbb{R}^{K \times 768}$ to $\mathbb{R}^{768}$. To achieve this, we apply mean pooling across the token dimension. The resulting tweet embedding is defined as:

$$e_i = \frac{1}{k_i} \sum_{j=1}^{k_i} \mathbf{h}_{i,j}, \quad e_i \in \mathbb{R}^{768}.$$

This produces a length-768 vector reflecting the aggregate semantic content of the tweet, which can be thought of as "center of mass" of the contextualized token representations. After pooling, the tweet-level embeddings from all N tweets are stacked into a matrix:

$$E = \begin{bmatrix} e_1 \\ \vdots \\ e_N \end{bmatrix} \in \mathbb{R}^{N \times 768}.$$

where each row corresponds to a single tweet's embedding. In our dataset, this results in $N = 248{,}915$ embeddings, each of dimension 768, forming the input feature matrix for our virality classifier.

All embeddings were computed on an Apple Silicon device using the `mps` backend. These were computed in batches of size 1-2 due to memory constraints. Extraction rate averaged 3.2 tweets per second, resulting in a 12-hour full extraction process.
The final embedding tensor was saved as:

`bertweet_embeddings.pt`.

3.5 Metadata Features

In addition to the BERTweet text embeddings, we incorporate six non-textual metadata features known to correlate with tweet engagement. The metadata features are:

- `followers`: number of followers of the author,
- `favs`: number of likes,
- `retweets`: number of retweets,
- `quotes`: number of quote-tweets,
- `positive`: positive sentiment score (0–1),

- `negative`: negative sentiment score (0–1).

These features were extracted directly from the dataset without any normalization beyond converting missing values to zero. The features were then concatenated with the BERTweet embedding to produce a hybrid input representation for each tweet.

3.6 Classification Model

To model tweet virality from text alone, we trained a lightweight neural classifier over the BERTweet embeddings. Each input tweet was first tokenized and then fed into the encoder to obtain contextualized token embeddings. The classifier consisted of:

- a dropout layer ($p = 0.2$),
- a linear layer mapping $\mathbb{R}^{768}$ to the three virality classes.

The model was optimized using Adam (learning rate 2×10^{-5}) for 3 epochs with batch size 32. The loss function was cross-entropy, and all training was performed using PyTorch on the MPS backend.

3.7 Computational Notes and Reproducibility

All preprocessing steps—including text extraction, cleaning, virality labeling, and embedding generation—were condensed in a scripted pipeline to ensure full reproducibility. The pipeline can be executed end-to-end when implemented in Python.

3.8 Hybrid Feature Construction

For the hybrid model, the metadata features are concatenated to the tweet's 768-dimensional BERTweet embedding vector. For tweet i, let $m_i \in \mathbb{R}^6$ denote the metadata feature vector. The hybrid representation is:

$$h_i = [\, e_i \,\|\, m_i \,] \in \mathbb{R}^{774}$$

where $\|$ denotes vector concatenation, and $774 = 768$ embeddings + 6 metadata features. This enlarged feature vector is then passed to the same multinomial logistic regression classifier architecture as in the text-only model.

3.9 Logistic Regression Classifier on BERTweet Embeddings

We train two versions of a multinomial regression classifier to predict virality class labels: a text-only model and a hybrid model. The logistic regression model gives an interpretable, computationally efficient baseline for assessing how much signal exists in solely text embeddings, without further deep finetuning.

3.9.1 Train–Test Split

Let $E \in \mathbb{R}^{N\times 768}$ denote the matrix of tweet embeddings and $y \in \{0,1,2\}^N$ the corresponding virality labels. To account for class imbalance, we performed a stratified 80/20 train-test split:

$$(E_{\text{train}}, E_{\text{test}}, y_{\text{train}}, y_{\text{test}}) = \text{StratifiedSplit}(E, y, \text{test}_{\text{size}} = 0.2)$$

3.9.2 Class Weighting

Since the total training size was almost 200,000 tweets, it was important to ensure the model did not default to predicting low virality because of the imbalance in virality classes. Thus we applied class weights in the cross-entropy objective. For class $c \in \{0,1,2\}$, and $\text{count}(N) =$ the size of each class,

$$w_c = \frac{N}{3 \cdot \text{count}(N)}$$

which increases the penalty for misclassifying underrepresented classes and stabilizes training.

3.9.3 Model Formulation

The classifier is a single linear layer,

$$f(x_i) = Wx_i + b, \quad W \in \mathbb{R}^{3\times 768}, b \in \mathbb{R}^3$$

where x_i is either the 768-dimensional text-only embedding e_i or the 774-dimensional hybrid vector h_i.

where f takes the tweet-level embedding vector $x_i \in \mathbb{R}^{768}$ as an input and returns an output of three classes, corresponding exactly to multinomial logistic regression with a learned weight matrix and bias vector. Given logits $z_i = f(e_i)$, the predicted probability of class c is given by the softmax:

$$p(y_i = c \mid e_i) = \frac{e^{z_i,c}}{\sum_{j=0}^{2} e^{z_i,j}}$$

The *softmax* function, or the *normalized exponential*, represents a smoothed version of the 'max' function (1).

3.9.4 Training

We optimize the weighted cross-entropy loss

$$\mathscr{L} = -\sum_{i=1}^{N_{\text{train}}} w_{y_i} \log\, p(y_i \mid e_i))$$

using the Adam optimizer with a learning rate of 10^{-3}.

The model is trained for 8 epochs over the full training set, with the embeddings treated as fixed input features (i.e., no gradient flow into BERTweet).

3.9.5 Model Evaluation

Two predictive models were developed and compared:

1. Text-only model using 768-dimensional BERTweet embeddings.
2. Hybrid model combining those embeddings with six additional metadata features (followers, favorites (likes), retweets, quotes, positive sentiment, negative sentiment).

Both models were trained on identical splits of the dataset (199,132 training examples and 49,783 test examples), using weighted cross-entropy to correct for class imbalance.
After training, predictions were obtained by

$$\hat{y}_i = \arg\max_c p(y_i = c \mid e_i)$$

and evaluated on the test set.

4 Results

We report the overall test accuracy, macro-averaged precision, recall, F1 score, the full per-class classification report, and the confusion matrix.
Training set class weights (empirically computed) were:

```
Class 0: 0.4232  # Very common, down-weighted
Class 1: 2.5711  # Much less frequent, up-weighted
Class 2: 4.0329  # Smallest class, heavily up-weighted
```

4.1 Text-Only Model

Using frozen BERTweet embeddings (768-dimensional) and a weighted multinomial logistic regression classifier, the text-only model achieved:

```
Test accuracy: 0.6885
Macro F1: 0.394
Weighted F1: 0.685
```

These numbers reflect the model's demonstrated ability to distinguish *non-viral* tweets (Class 0), while performance on higher-virality classes is weaker, which is expected given the underlying class imbalance.

4.1.1 Training Loss by Epoch

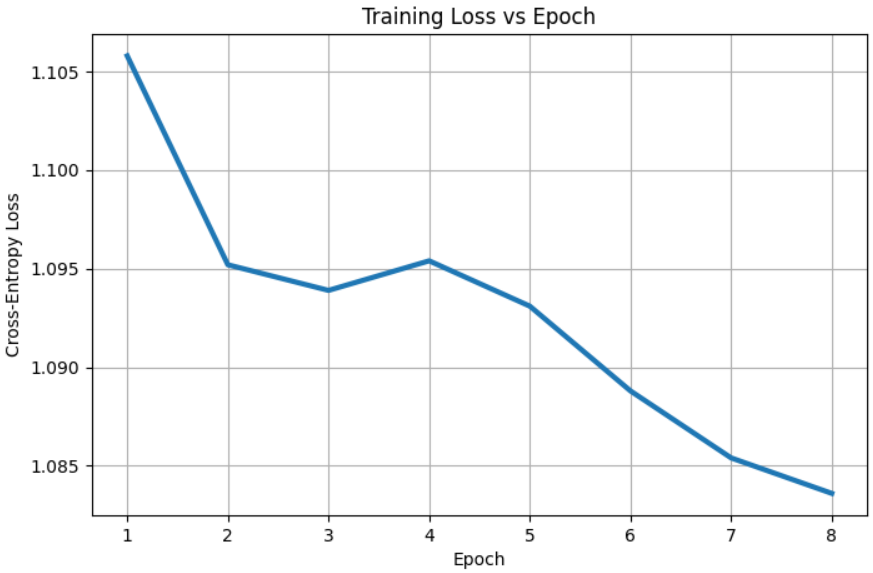

Figure 1: Training Loss by Epoch: Text-only

4.1.2 Classification Report & Confusion Matrix

```
Classification Report (Macro & Per-Class):
              precision    recall  f1-score   support

           0     0.8266    0.8250    0.8258     39214
           1     0.1611    0.0879    0.1137      6454
           2     0.1908    0.3305    0.2419      4115

    accuracy                         0.6885     49783
   macro avg     0.3928    0.4144    0.3938     49783
weighted avg     0.6878    0.6885    0.6852     49783

Confusion Matrix:
[[32350  2530  4334]
 [ 4453   567  1434]
 [ 2333   422  1360]]
```

4.2 Hybrid Model

The hybrid model achieved higher overall performance:

```
Test accuracy: 0.7208
Macro F1: 0.3696
Weighted F1: 0.6921
```

Although the macro F1 remains similar, overall test accuracy improves by 3.2 percentage points, suggesting a better alignment with the true class distribution.

4.2.1 Training Loss by Epoch

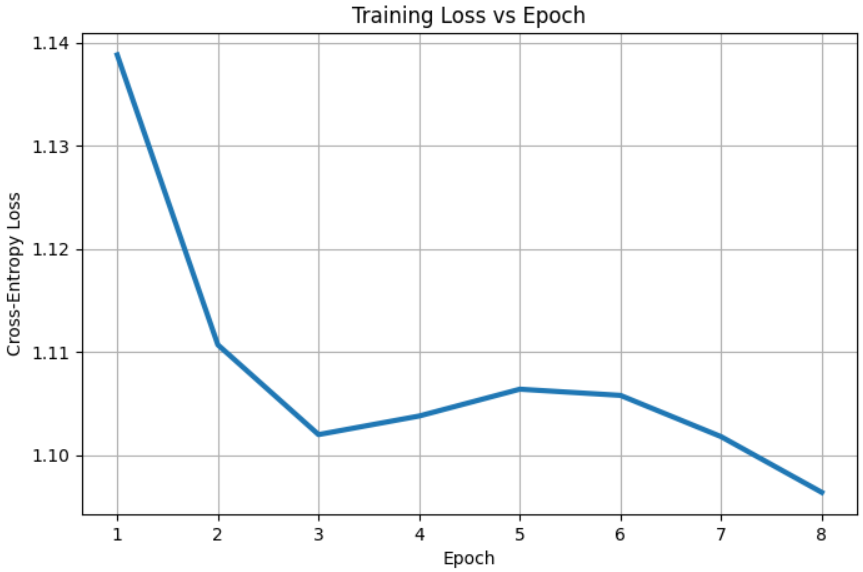

Figure 2: Training Loss by Epoch: Hybrid Model

4.2.2 Classification Report & Confusion Matrix

```
Classification Report (Macro & Per-Class):
              precision    recall  f1-score   support

           0     0.8037    0.8862    0.8430     39214
           1     0.1771    0.1029    0.1302      6454
           2     0.1679    0.1140    0.1358      4115

    accuracy                         0.7208     49783
   macro avg     0.3829    0.3677    0.3696     49783
weighted avg     0.6699    0.7208    0.6921     49783

Confusion Matrix:
[[34753  2558  1903]
 [ 5368   664   422]
 [ 3119   527   469]]
```

5 Interpretation

5.1 Loss & Test accuracy

1. **Text-only Model:** Loss starts around 1.10, and gradually decreases to 1.0836. 68.85% of test tweets were assigned the correct virality class.

2. **Hybrid Model:** Loss starts around 1.14, and gradually decreases to 1.0964. 72.08% of test tweets were assigned the correct virality class.

In both cases, the change in loss is small but stable, indicating that the model is learning. The test accuracy is also heavily influenced by Class 0 (non-viral) tweets.

5.2 Classification Report Breakdown: Text-only Model

5.2.1 Class 0:

`precision:` `0.8266` → the model predicts "non-viral" correctly 82.66% of the time.
`recall:` `0.8250` → the model correctly identifies 82.50% of actual non-viral tweets.
`f1-score:` `0.8258` → harmonic mean of precision & recall.
`support:` `39,214` → the number of Class 0 examples in the test set.
⇒ This class dominates performance.

5.2.2 Class 1:

`precision:` `0.1611` → only 8.79% of class-1 tweets are correctly detected.
`recall:` `0.0879` → predictions of class 1 are incorrect approximately 84% of the time.
`f1-score:` `0.1137` → low due to both precision and recall being weak.
`support:` `6,454` → the number of Class 1 examples in the test set.
⇒ These tweets are rare and hard to distinguish.

5.2.3 Class 2:

`precision:` `0.1908` → most predictions of class 2 are incorrect.
`recall:` `0.3305` → the model finds 33% of viral tweets.
`f1-score:` `0.2419` → modest because recall > precision.
`support:` `4,115` → the number of Class 2 examples in the test set.
⇒ The model struggles to identify viral tweets, correctly classifying only 33% of them while misclassifying most as non-viral.

5.3 Classification Report Breakdown: Hybrid Model

5.3.1 Class 0:

`precision:` `0.8037` → the model correctly predicts "non-viral" 80.37% of the time.
`recall:` `0.8862` → the model correctly identifies 88.62% of actual non-viral tweets.
`f1-score:` `0.8430` → high due to strong recall and solid precision.

`support: 39,214` → the number of Class 0 examples in the test set.
⇒ Class 0 performance improves in recall compared to text-only, suggesting metadata helps identify ordinary/non-viral tweets more reliably.

5.3.2 Class 1:

`precision: 0.1771` → about 17.71% of tweets are correctly predicted as class 1.
`recall: 0.1029` → the model captures only 10.29% of actual class-1 tweets.
`f1-score: 0.1302` → low due to both precision and recall being weak.
`support: 6,454` → the number of Class 1 examples in the test set.
⇒ Class 1 remains the hardest class, but precision improves slightly compared to text-only. Metadata does not appear to give the model enough signal to discern "moderately viral" tweets.

5.3.3 Class 2:

`precision: 0.1679` → most predictions of viral tweets are incorrect.
`recall: 0.1140` → the model identifies only 11.40% of actual viral tweets.
`f1-score: 0.1358` → modest because recall > precision
`support: 4,115` → the number of Class 2 examples in the test set.
⇒ Class 2 gets worse compared to the text-only model. The hybrid model becomes more conservative, assigning fewer tweets to the viral class.

5.4 Macro/Weighted Metrics

```
# Text-only:
accuracy:      0.6885
macro avg F1:  0.3938
weighted avg:  0.6852

# Hybrid:
accuracy: 0.7208
macro avg F1: 0.3696
weighted avg: 0.6921
```

Accuracy improves substantially, from 68.85% to 72.08%, which is driven entirely by an improved detection of class-0 tweets. However, Macro F1 drops (0.3938 → 0.3696), showing worse performance on minority classes (1 and 2). The weighted F1 rises slightly (0.6852 → 0.6921), again, this is because class 0 dominates.

Computing two different F1-scores is useful as it paints a broader picture of how our model is treating tweets belonging to different classes. A macro F1 averages F1 across classes, treating them equally, punishing low performance on classes 1 & 2. The weighted F1 is weighted by class size— here, the dominance of class 0 is reflected in how much higher this F1 score is compared to the macro average. In the context of our contrasting models, analyzing macro vs. weighted F1 reveals a key tradeoff: meta-features help overall accuracy by making the model even better at class 0, but they negatively affect balance across classes.

5.5 Confusion Matrix Breakdown

Each row in the confusion matrix represents the actual class to which a tweet belongs, and each column represents the predicted class. Going row by row:

5.5.1 Row 0 (actual: non-viral)

```
# Text-only:
[32350, 2530, 4334]
# Hybrid:
[34753, 2558, 1903]
```

In the text-only model, 32,350 tweets are correctly labeled non-viral, 2,530 tweets are mislabeled moderately viral, and 4,334 tweets are mislabeled highly viral. In the hybrid model, 34,753 tweets are correctly classified as non-viral, 2,558 tweets are mislabeled as moderately viral, and 1,903 tweets are misclassified as highly viral.

The text-only model sometimes places ordinary tweets into viral categories. In comparison, the hybrid model becomes more conservative, assigning far fewer non-viral tweets into viral classes. We interpret this as metadata helping the model better identify ordinary/non-viral content.

5.5.2 Row 1 (actual: moderately viral)

```
# Text-only:
[4453, 567, 1434]
# Hybrid:
[5368, 664, 422]
```

In the text-only model, 567 tweets are correctly predicted as moderately viral, 4,453 tweets are predicted as non-viral, and 1,434 tweets are confused as highly viral. In the hybrid model, 664 tweets are correctly predicted as moderately viral, 5,368 tweets are misclassified as non-viral, and 422 tweets are incorrectly labeled as highly viral.
$\Rightarrow$ Moderately viral tweets seem to behave ambiguously due to belonging to a middle ground of virality. The addition of metadata increases a tendency of moderately viral tweets to collapse into class 0, suggesting the hybrid model interprets metadata distributions for class 1 as being more similar to class 0 than to class 2.

5.5.3 Row 2 (actual: highly viral)

```
# Text-only:
[2333, 422, 1360]
# Hybrid:
[3119, 527, 469]
```

In the text-only model, 1,360 tweets are correctly predicted as viral, 2,333 tweets are incorrectly predicted as non-viral, and 422 tweets are mislabeled as moderately viral. In the

hybrid model, 469 tweets are correctly predicted as highly viral, 3,119 tweets are misclassified as non-viral, and 527 tweets are mislabeled as moderately viral.
⇒ Highly viral tweets can often textually appear to be ordinary tweets. These tweets become even harder for the hybrid model to correctly identify. The model shifts toward under-predicting class 2, suggesting that metadata features may not strongly distinguish highly viral tweets in this dataset, possibly even making them resemble ordinary tweets.

5.6 Discussion

The text-only model has a superior recall for Class 2 (high virality) relative to the hybrid model, suggesting that text content plays a key role in discerning highly viral tweet, likely because the textual style matters most once a tweet breaks through the noise. This makes the text-only model more revealing about what kinds of language "go viral."

The hybrid model integrates structural and behavioral indicators of influence, such as follower count (potential reach), retweets, likes, and sentiment (emotional charge). These features allow the model to better classify non-viral tweets. Realistically, most accounts produce tweets that receive limited interaction; metadata helps the classifier identify these with high accuracy. However, we note that the hybrid model becomes overly conservative, rarely predicting high-virality outcomes. In other words, the hybrid model is more confident in saying "this tweet will not go viral" than identifying tweets that will.

6 Limitations

While this study does provide some evidence that linguistic content contributes meaningfully to tweet virality prediction, we identify several limitations regarding the strength and generality of the findings.

1. There is a methodological limitation in our hybrid model: it includes certain engagement metrics (retweets, quotes) as input features when raw virality is defined from these same metrics. This creates partial circularity, as we are predicting from the outcome itself. This setup is applicable to scenarios where early engagement signals are available and we wish to forecast continued growth (see Section 8). Future work should separate signals (such as engagement in the first couple of hours after a tweet is posted) from final outcomes to properly isolate predictive factors.

2. The virality labels are imbalanced. By consolidating various engagement metrics into three categories, we reduce complexity of the actual learning task, but this can also veil important differences between moderately viral posts and significantly viral posts. The fact that most tweets exhibit very little engagement leads to a distribution that is skewed. Consequently, models trained using this distribution have fewer samples available for high-virality training examples, making them less stable.

3. Although the hybrid model integrates basic tweet metadata (follower count, favorites, retweets, sentiment), it still omits key structural and network-level information known to influence virality, such as the follower counts of retweeters. The hybrid model's

higher accuracy largely reflects improved detection of non-viral tweets, but its performance on moderately and highly viral tweets remains weak. This indicates that our metadata does not capture amplification mechanisms such as retweet cascades or community structure. As prior work shows, these factors are often the strongest predictors of virality, and their absence possibly limits the model's ability to distinguish between moderate and high virality.

4. Lastly, while this dataset is considerably sized, it only captures a single point-in-time view of Twitter content and cannot therefore be generalized over time or across topics. In addition, language, memes, and social media language patterns and user behavior evolve continuously, which implies that the predictive power of models created from historical data grows weaker when these models are applied to existing data.

We investigated whether longer optimization would reduce training loss and improve generalization. Increasing epochs to 50/100 reduced training loss further but substantially degraded test accuracy (40–59%), indicating overfitting and instability. To address this, we implemented a mini-batch training protocol with a validation split, early stopping, weight decay, and learning-rate tuning. This controlled setup prevents overfitting and produces stable validation performance; given observed per-epoch timings ($\approx$0.1–1.5s for linear models, $\approx$1–2s for modest MLPs), exploring up to 30–50 epochs is computationally feasible (minutes of wall time). We therefore recommend rerunning longer training with the improved protocol; otherwise, reporting results from the original 8-epoch run with an explanation of feasibility and instability is appropriate.

7 Conclusion

We sought to understand whether virality can be determined by tweet text alone, and to what extent simple metadata contributed to these effects. After using BERTweet embeddings combined with lightweight classifiers, we concluded that language indeed has a virality signal that can be extracted from the tweet. We discovered that the text-only method has approximately 69% accuracy, significantly better than random baseline results (as discussed in Section 10), showing that tweets have stylistic and semantic relationships to their eventual reach. Metadata improves predictions of tweets that will not result in virality, but has less impact on the predictions of tweets that do go viral. The addition of metadata increases accuracy from 69% to 72%, yet at the same time, makes the models more conservative and less likely to identify very viral tweets. High-virality predictions require information other than textual content.

Structural determinants influence amplification on social networks via follower networks, retweet cascades, and visibility dynamics; therefore, neither the textual content nor the basic tweet metrics are able to account for all structural elements. The addition of more training epochs reduces rather than increases generalization ability; this is due to an effective model overfitting and a limitation in the model's ability to extract more signal through the use of frozen embeddings.

Key findings from this study continue to emphasize both the potential and limits of a purely text-based virality prediction method. Transformer embeddings produce acceptable

linguistic support for predictive performance; however, successful prediction of high-virality tweets will require additional social-network related features/variables beyond textual information alone.

8 Future Work

One way to continue this research would be to model virality as a temporal diffusion process using Markov chains or agent-based simulations. Instead of predicting final engagement from static textual or metadata features, this framework would capture how tweets move from dormant to trending to viral states over time. Each transition probability would depend on BERTweet text embeddings and network structure features such as follower graphs. This would enable us to probabilistically forecast cascade dynamics within a given network, and address the question of how linguistic content interacts with network structure to accomplish virality. Agent-based simulations could further model individual retweet decisions as stochastic processes, revealing the intricate relationships between tweets and retweets.

9 Acknowledgement

I would like to thank Professor Nicholas (Nick) Moore for valuable guidance throughout the development of this project, the Mathematics Department at Colgate University for providing computational resources and institutional access to journals and textbooks, and my academic advisor Professor William (Will) Cipolli for letting me use his dataset for this project. I am also grateful to Lola Garrigue, Leonardo Cambisaca, and Dennis Belotserkovskiy, for their feedback and support during all stages of this research.

References

[1] Bishop, C. M. (2006). *Pattern recognition and machine learning.* Springer.

[2] Nguyen, D. Q., Vu, T., & Nguyen, A. T. (2020). BERTweet: A pre-trained language model for English tweets. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations.* https://doi.org/10.48550/arXiv.2005.10200

[3] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. *MIT Press.*

[4] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems.*

[5] Hasan, R., Cheyre, C., Ahn, Y.-Y., Hoyle, R., & Kapadia, A. (2022). The Impact of Viral Posts on Visibility and Behavior of Professionals: A Longitudinal Study of Scientists on Twitter. *Proceedings of the International AAAI Conference on Web and Social Media*, 16(1), 323-334. https://doi.org/10.1609/icwsm.v16i1.19295

[6] Jenders, M., Kasneci, G., & Naumann, F. (2013). Analyzing and predicting viral tweets. *Proceedings of the 22nd International Conference on World Wide Web Companion (WWW '13 Companion)*, 657–664. https://doi.org/10.1145/2487788.2488017

[7] Subbian, K., Prakash, B. A., & Adamic, L. (2017). Detecting large reshare cascades in social networks. *Proceedings of the 26th International Conference on World Wide Web (WWW '17)*, 597–605. https://doi.org/10.1145/3038912.3052718

[8] Berger, J., & Milkman, K. L. (2012). What makes online content go viral? *Journal of Marketing Research*, 49(2), 192–205.

[9] Stieglitz, S., & Dang-Xuan, L. (2013). Emotions and information diffusion in social media—Sentiment of microblogs and sharing behavior. *Journal of Management Information Systems*, 29(4), 217–248.

[10] Romero, D. M., Meeder, B., & Kleinberg, J. (2011). Differences in the mechanics of information diffusion across topics: Idioms, political hashtags, and complex contagion on Twitter. Proceedings of the 20th International Conference on World Wide Web, 695–704.

[11] Cheng, J., Adamic, L., Dow, P. A., Kleinberg, J. M., & Leskovec, J. (2014). Can cascades be predicted? Proceedings of the 23rd International Conference on World Wide Web, 925–936.

10 Appendix

To contextualize model performance, we also computed a random-guessing baseline in which predictions were sampled uniformly from $\{0, 1, 2\}$. This provides a lower bound for chance-level performance.

```
##### DUMMY CASE #####
import numpy as np
from sklearn.metrics import accuracy_score
np.random.seed(0)
random_preds = np.random.choice([0,1,2], size=len(y_test))
print("Random baseline accuracy:", accuracy_score(y_test, random_preds))

# Result:
Random baseline accuracy: 0.33256332482976114
```